



FRIEDRICH-SCHILLER-
UNIVERSITÄT
JENA

Turning Python scripts built with Sugar into web apps

Tom Eulenfeld, Maria Schreiber
Friedrich Schiller University Jena
Workshop, GCB 2026

RNA
BIOINFORMATICS
& HIGH-THROUGHPUT ANALYSIS

Before we start...

A Hands on: Preparation (Install conda environment, download data)

1. Introduction to sugar

B Hands on: Use Sugar and MAFFT to prepare and plot an alignment

2. NiceGUI Concepts

C Hands on: Build a web app to upload and plot an alignment

D Hands on: Extend web app (options, download)

3. Plotting “neighbor” genes of a feature

E Hands on: Prepare files for “NextGene” app and plot features

4. NiceGUI: long running tasks, accessing app on own server

F Hands on: Implement NextGene app

Wrap-up & Feedback

Instructions in the workshop repository:

https://github.com/rnajena/workshop_gcb2026_webapps

- Clone or download repo to get files
- Prepare conda environment
- Find this presentation



A Python framework for bioinformatics



The sugar project aims to provide a lightweight framework to facilitate rapid application development for bioinformatics.

```
pip install rnajena-sugar
```

```
from sugar import read
seqs = read("some_seqs.fasta")
seqs.translate()
seqs.write("some_aas.fasta")
```



<https://github.com/rnajena/sugar>

Check out documentation and tutorial

- Input / output of seq files with format auto-detection
- Plugin interface for IO

```
seqs = read('NC_045512.2.gb')
print(seqs)
seqs2 = read('AF086833.fasta')
print(seqs2)
```

```
1 seqs in basket
NC_045512  30k  ATTAAGGTTTATACCTTCCCAGGTAACAAACCAACCACTTTTCGATC...  GC:37.97%
1 seqs in basket
AF086833.2  19k  CGGACACACAAAAAGAAAGAAGAATTTTTAGGATCTTTTGTGTGCGA...  GC:41.07%
```

format	module	read	write	description
fasta	<code>sugar._io.fasta</code>	✓	✓	FASTA IO
genbank	<code>sugar._io.genbank</code>	✓		GenBank reader
stockholm	<code>sugar._io.stockholm</code>	✓	✓	Stockholm IO
gff	<code>sugar._io.gff</code>	✓	✓	Generic feature format (GFF) and Gene transfer format (GTF) IO
sjson	<code>sugar._io.sjson</code>	✓	✓	SJson IO, custom lossless sugar format

```
seqs = read('NC_045512.2.gb')
seqs2 = read('AF086833.fasta')
print(seqs2[0].meta)
print(seqs[0].meta)
```

```
id: AF086833.2
_fasta: Attr(header='AF086833.2 Ebola virus
_fmt: fasta

id: NC_045512
_fmt: genbank
_genbank: Attr(locus='NC_045512, 29903, bp,
features:
    source      0+ 29_903   seqid=NC_045512;
    5' UTR      0+   265   seqid=NC_045512;
    gene        265+ 21_290 seqid=NC_045512;
    CDS         265+ 13_203 seqid=NC_045512;
                13_467+ 8_088
mat_peptide     265+   540   seqid=NC_045512;_genbank=Attr(gene='ORF1ab', locu...
```

```
seqs3 = read('ali_pestivirus.stk')
print(seqs3[:3, 200:220])
seqs3.str.replace('-', '')
seqs3.write('pestivirus_original.fasta')
```

1 seqs in basket
NC_045512 7096 MESLVPGFNEKTHVQLSLPVLQVRDVLVRGFGDSVEEVLSEARQHLKDGTCLVEVEKG...

3 seqs in basket

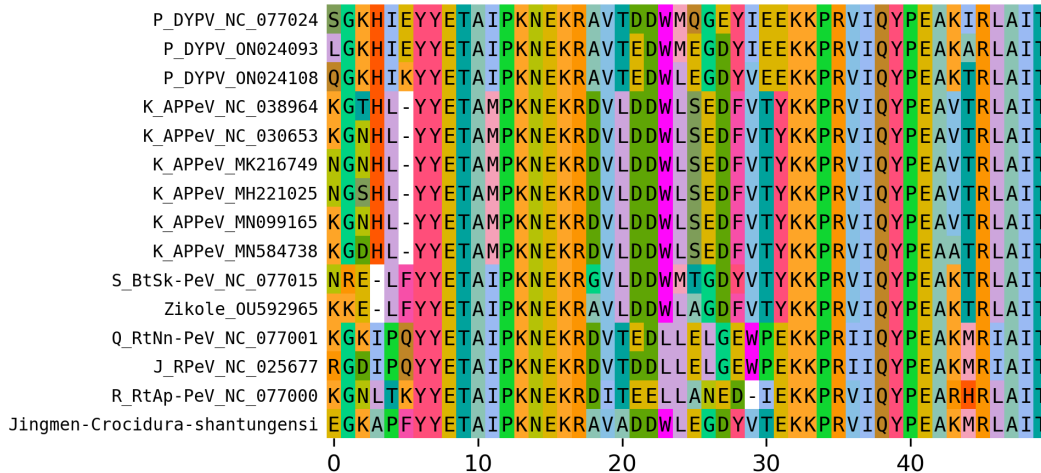
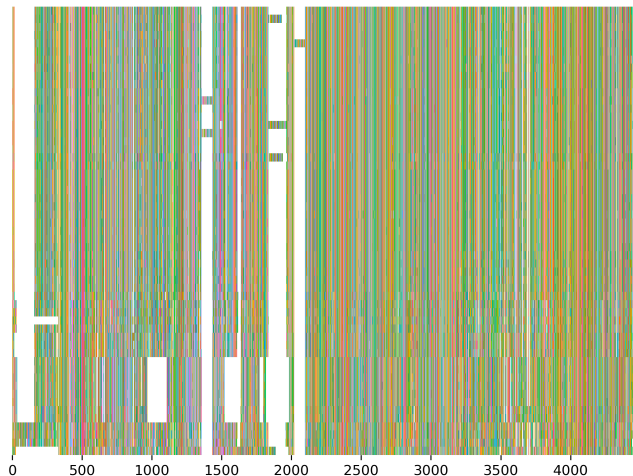
A_BVDV1_NC_076029	20	GACTAGC-----ATAATG	GC:38.46%
A_BVDV1_NC_001461	20	GACTAGC-----ATAATG	GC:38.46%
A_BVDV1_KC853440	20	GACTAGC-----AAACG	GC:50.00%



• Plot alignment

```
seqs3 = read('ali_pestivirus_aa.clustal')
seqs3.plot_alignment(show=True)
seqs3.plot_alignment(fname='ali_pestivirus.png')

seqs3[-15:, 4000:4050].plot_alignment(show=True)
seqs3[-15:, 4000:4050].plot_alignment(fname='ali_pestivirus_subset.png')
```



- Input / output of feature files with format auto-detection

```
from sugar import read_fts
fts = read_fts('AF086833.gb')
print(fts[:6])
fts[:6].write('AF086833_test.gff')
fts_ia = read_fts('influenza_a_blast.tsv')
seqs_ia = read('influenza_a.fasta')
print(seqs_ia[fts_ia])
```

```
source    0+ 18_959  seqid=AF086833;_genbank=Attr(o
5'UTR     0+      55  seqid=AF086833;_genbank=Attr(n
gene      55+   2_971  seqid=AF086833;_genbank=Attr(g
mRNA      55+   2_971  seqid=AF086833;_genbank=Attr(g
regulatory 55+    12    seqid=AF086833;_genbank=Attr(r
CDS       469+  2_220  seqid=AF086833;_genbank=Attr(g
```

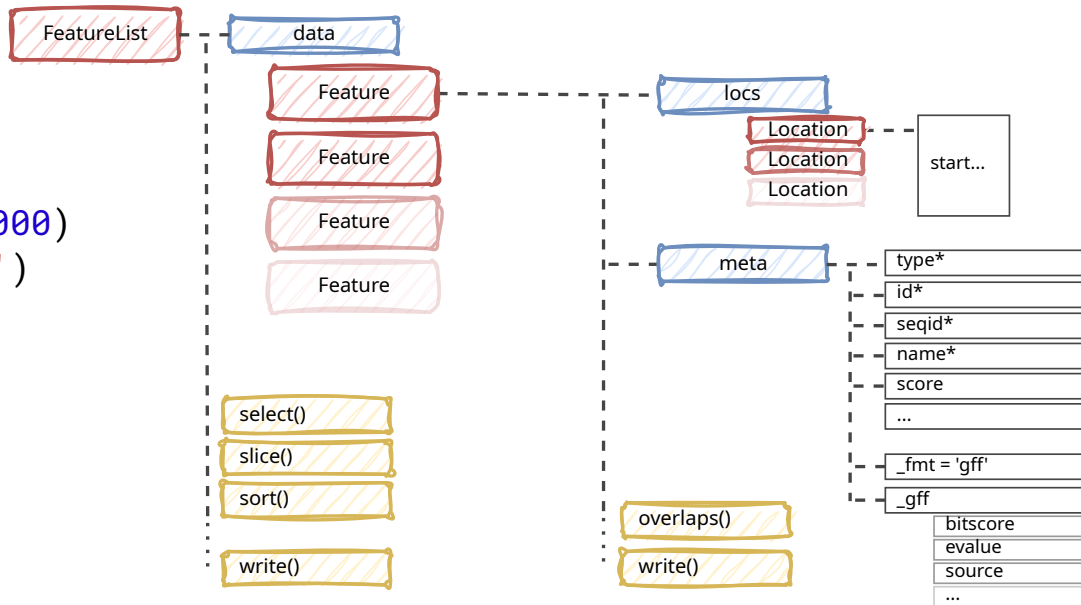
10 seqs in basket

```
LC817404.1 10 TAAAAGAACT GC:20.00%
LC817404.1 9  TAAAAGAAC  GC:22.22%
LC817404.1 8  AAAAGAAC   GC:25.00%
...
```

format	module	read	write	description
gff	<code>sugar._io.gff</code>	✓	✓	Generic feature format (GFF) and Gene transfer format (GTF) IO
gtf	<code>sugar._io.gff</code>	✓	✓	Generic feature format (GFF) and Gene transfer format (GTF) IO
genbank	<code>sugar._io.genbank</code>	✓		GenBank reader
infernai	<code>sugar._io.tab.infernai</code>	✓		Infernal reader for output generated with tblout fmt 1, 2, 3
mmseqs	<code>sugar._io.tab.mmseqs</code>	✓		MMseqs2 reader for output generated with option fmtmode 4 (preferred) or 0
blast	<code>sugar._io.tab.blast</code>	✓		BLAST reader for output generated with outfmt 7 (preferred), 6 or 10
tsv	<code>sugar._io.tab.xsv</code>	✓	✓	TSV, CSV and XSV file formats IO
csv	<code>sugar._io.tab.xsv</code>	✓	✓	TSV, CSV and XSV file formats IO
sjson	<code>sugar._io.sjson</code>	✓	✓	SJson IO, custom lossless sugar format

- Feature objects have their own methods

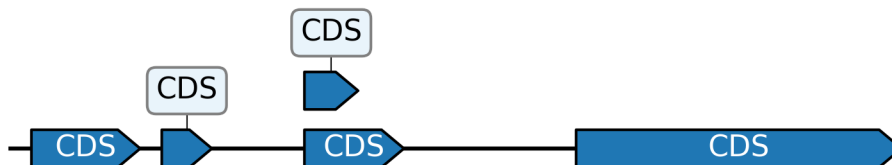
```
from sugar import read_fts
fts = read_fts('AF086833.gb')
fts2 = fts.select('cds', len_ge=1000)
fts2.write('AF086833_long_cds.gff')
```



- Plot features

```
fts2.plot_ftsvviewer(show=True)  
fts2.plot_ftsvviewer(fname='AF086833_long_cds.png')
```

AF086833



- Selection of other useful features
 - Index and query FASTA files
 - Download sequences via entrez web API
 - Command line tool for common tasks
 - ...

B Hands On: Use Sugar and MAFFT to prepare and plot an alignment

<https://rnajena-sugar.readthedocs.io>

Switch to the working directory wd !

- a) Read the BLAST and MMSeqs2 files in `../data/hok/` with `read_fts`
- b) Read the corresponding genomes in `../data/hok/genomes/*/` with `read`
- c) Select the subsequences corresponding to the hits
- d) Write these to a new file
- e) Switch to command line and align with `mafft --auto in.fasta > out.aln`
- f) Read and plot the alignment file, use the `option` `aspect=2`

Bonus: Generate a second alignment with the protein templates

`../data/hok/query_hok.fasta` and the translated BLAST and MMSeqs2 hits. Show sequence ID and start position as label in the alignment plot by changing each sequence ID to include the start location from the corresponding features `locs.start` attribute.

```
from nicegui import ui
```

```
@ui.page('/') ←
```

Decorator to mark pages

```
def main():
```

```
    def greet(event):
```

```
        label.set_text(f"Hello {event.value}!")
```

```
    ui.label('Greeting').classes('text-2xl font-bold')
```

```
    with ui.row().classes('items-center'):
```

```
        ui.label('Please enter your name:')
```

```
        user_input = ui.input(on_change=greet)
```

```
    label = ui.label()
```

Declarative layout using
with and ui objects

Styling with .classes()
and .props() methods

```
ui.run(title='Hello NiceGUI')
```

Start the server

2 NiceGUI concepts we need in a moment

- Server setting: Either work in memory or with temporary files
- To create a BioBasket object from memory use: `from sugar import BioBasket`
`seqs = BioBasket.fromfmtstr(content)`
- See nicegui doc for the correct `async-await` pattern for the `handle_upload` function
- To plot the alignment use the `ui.matplotlib().figure` context (see doc)
- The alternative using `ui.Image` can be done in multiple steps
 1. Plot to a temporary file or to an in-memory bytes object with `.plot_alignment`
 2. Convert these bytes to a base64 image string
 3. Display the image by updating the image source with `set_source`

C Hands On: Build a web app to upload and plot an alignment

<https://nicegui.io/documentation/>
<https://rnajena-sugar.readthedocs.io>



- Write a function `aliplot(content, fig)` to plot an alignment described by the (FASTA) string in a matplotlib figure
- Create `upload` and `matplotlib().figure` UI elements in the main page function
- Call `aliplot` on finished upload

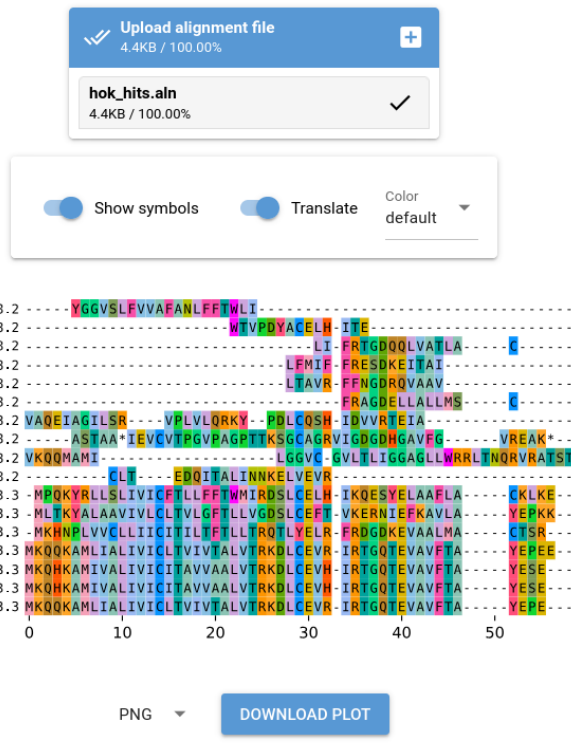


We continue 3pm ...

D Hands On: Extend web app (options, download)

<https://nicegui.io/documentation/>
<https://rnajena-sugar.readthedocs.io>

Alignment Viewer



- Add ui elements to change some options of the plot (e.g. translate nucleotides to amino acids, show/hide symbols/labels, change color scheme), corresponding options need to be introduced to the `aliplot` function
- Replot alignment, when options change (for this we need the previously uploaded data, save the uploaded content in a dict defined in the page function scope)
- Add dropdown `ui.select` for format and a download button
- When clicked, the app should save the figure to a bytes object which can be passed to `ui.download`

D Hands On: Extend web app (options, download) – Helper code snippets

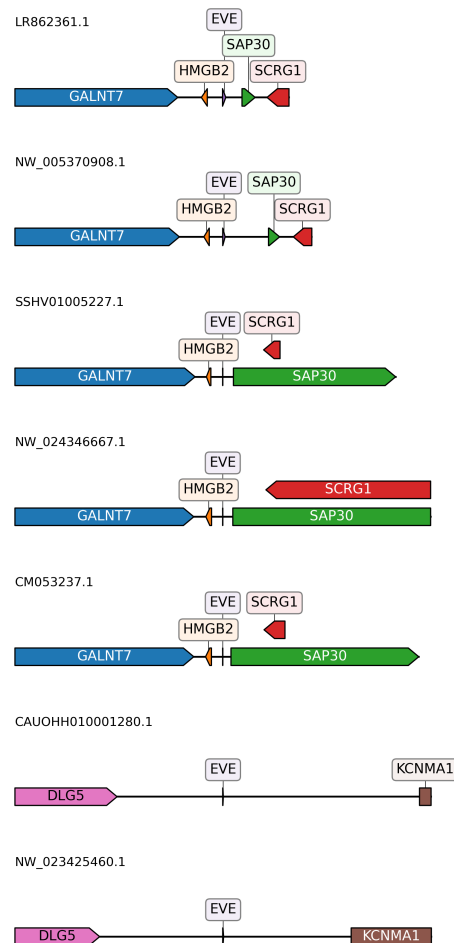
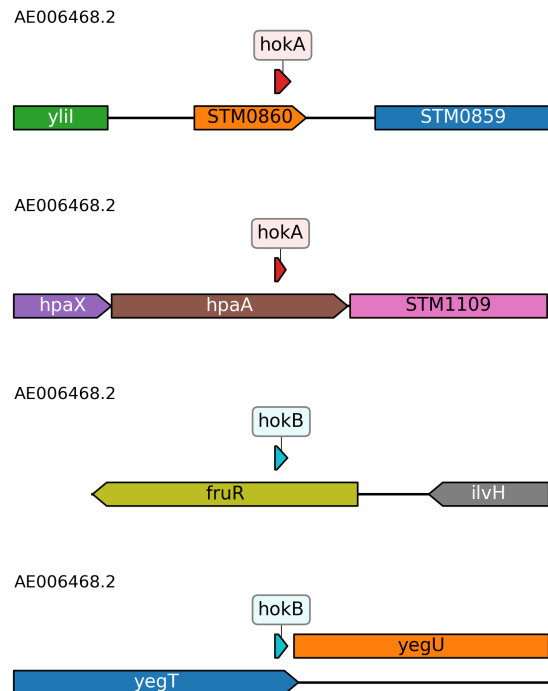
```
@ui.page('/')
def main_page():
    state = {'upload': None}

    async def handle_upload(event):
        content = await event.file.read()
        try:
            state['upload'] = content
            update_plot()
        except Exception as ex:
            ui.notify(f'Could not plot alignment: {ex}', type='negative')
            state['upload'] = None

import io

def fig_to_bytes(fig, **kwargs):
    buf = io.BytesIO()
    fig.savefig(buf, **kwargs)
    return buf.getvalue()
```

3 Plotting “neighbor” genes of a feature



E Hands on: Prepare files for “NextGene” app and plot features

<https://rnajena-sugar.readthedocs.io>

- a) Read the BLAST and MMSeqs2 output in `../data/hok/` and write a combined GFF file
- b) Read the two GFF files in `../data/hok/genomes/*/`, select all features with type gene and write to GFF
- c) Read the files from a) & b), create an empty FeatureList
- d) For each ft in a):
 - i. Select all features from b) with the same seqid and within a specific location range (e.g. `locrange=1000`) with `select` and `slice` methods
 - ii. Set `meta.group` for ft and all selected features to a unique integer
 - iii. Append those features to the growing FeatureList
- e) Plot the FeatureList and align a) features, use `options`, like `groupby='group'`, `align`, `colorby`

4. NiceGUI: await – async pattern and long running tasks

```
import time
from nicegui import run, ui

def compute_sum(a: float, b: float) -> float:
    time.sleep(1) # simulate a long-running computation
    return a + b

async def handle_click():
    result = await run.cpu_bound(compute_sum, 1, 2)
    ui.notify(f'Sum is {result}')

ui.button('Compute', on_click=handle_click)

ui.run()
```

4. NiceGUI: Local filesystem

- Deploying an app to the WWW?
 - Never expose the server's local filesystem to users.
 - Prefer controlled upload/download mechanisms.
 - Work within memory as much as possible.
 - Treat the server filesystem as private infrastructure, not user-accessible storage.
- If the app runs on your own, private server:
 - If the server is not accessible from outside, exposing the filesystem to yourself may be acceptable
 - Use a local file picker when convenient

4. NiceGUI: Accessing app run on own, private server

- Set `show=False` in `ui.run()`

```
ui.run(title='Alignment Viewer', show=False)
```

- Local port forwarding: Connect ports from server and your PC

```
tom@pc:$ ssh -L 8080:localhost:8080 server
```

- Start the app on the server

```
tom@server:$ python aliplot_webapp.py
```

- Open `http://localhost:8080/` in your browser@pc and start using the app

F Hands on: Implement NextGene app

NextGene Plotter

✓ Upload Genes Of Interest file 4.2KB / 100.00% +

hok_hits.gff ✓
1.1KB / 100.00%

✓ Upload Genes file 1.5MB / 100.00% +

hok_genes.gff ✓
1.5MB / 100.00%

Range: 10,000

—●—

PNG ▾

DOWNLOAD PLOT ↻

- Wrap the code from E Hands on in a function. It should take two (FASTA like) seq strings and return an image as a bytes string
- Add two `ui.upload` elements, one for the genes of interest and one for other genes, save the upload content in a dictionary defined in the page function scope
- Add a download button, the async download function should call a) via `cpu_bound` and download the returned bytes

Bonus: Add a slider to select the range, add a `ui.spinner` element to show when the app is busy creating the plot

Wrap up and Feedback

